Report – BST with Go

Gerald Fattah

CS378H – Assignment 3

October 22nd, 2025

Overview:

The goal of this project was to detect which binary search trees (BSTs) constructed from different input lines are equivalent. Meaning they contain the same values and would produce the same BST structure when inserted in the same order. The program supports a sequential version and several parallel versions that use goroutines, channels, locks, and worker pools to speed up hashing and comparison on larger inputs.

Sequential Algorithm:

The baseline implementation reads each line of input, inserts the integers into a BST, and then performs these steps:

1. In-order traversal – Produces a sorted slice of values for each tree.
2. Hashing – The traversal output is turned into a SHA-1 hash.
3. Grouping by hash – Trees with different hashes are guaranteed to be different; only matching hashes are potential duplicates.
4. Deep comparison – For trees that share a hash, a pairwise comparison confirms whether the trees are truly equivalent.
5. Union-Find – Equivalence classes are formed so the final output can list groups of identical trees.

This version runs entirely in one thread and serves as the correctness reference before adding concurrency.

Hash Grouping:

After hashing, the results must be grouped in a shared map keyed by the hash, which introduces synchronization. I implemented two different approaches to handle this step. The first is a channel-collector design, where workers send (hash, id) pairs to a single goroutine that updates the map. This avoids locks entirely and guarantees correctness through serialization, but it can also become a bottleneck because one goroutine must process all updates. The second approach uses multiple workers that update the map directly while holding a mutex. This removes the single choke point and can scale better when many workers are running, although threads may occasionally stall while waiting for

the lock. On small inputs, the two strategies perform similarly, but on larger inputs the trade-off becomes clear: the channel version is simple and smooth until the collector saturates, while the mutex version spreads the work out more evenly at the cost of increased contention.

Parallel Hashing

Hashing is an embarrassingly parallel task because each tree can be traversed independently. I implemented two hashing modes controlled by a flag:

- goroutine-per-tree – Spawns one goroutine for each tree. This is great for coarse workloads, but spawning thousands of goroutines on fine workloads adds overhead.
- worker pool – Uses a fixed number of hashing goroutines and sends tree indices through a channel. This avoids excessive goroutine creation and produces more stable timing on fine workloads.

Parallel Comparison:
Once hash grouping is done, only trees in the same hash bucket need deeper comparison. I support:

- goroutine mode – spawns a goroutine per comparison
- worker-pool mode – uses a bounded buffer plus a fixed number of workers

On simple.txt, everything finishes so fast that overhead dominates. On fine.txt, the worker pool is noticeably more consistent because it reuses goroutines instead of spawning hundreds or thousands of them.

Both modes update the shared Union-Find structure to form final equivalence classes.

Results and Observations

The performance of the goroutine and worker pool implementations was evaluated using three datasets: simple.txt, coarse.txt, and fine.txt.

For simple.txt, the tree sizes were very small, and the work per task was minimal. Because of this, the overhead of managing concurrency outweighed any parallel benefit. Execution times for both goroutine and pool implementations were nearly identical to the sequential

baseline. The graphs also showed irregular scaling, confirming that synchronization and task management costs dominated the runtime.

For coarse.txt, the larger trees made hashing and comparison operations expensive enough for concurrency to be worthwhile. Execution time dropped sharply as the number of hash workers increased, with clear speedup up to about four workers before leveling off. The goroutine implementation consistently ran slightly faster than the worker pool, showing lower scheduling overhead. Beyond four workers, both versions showed diminishing returns as the workload became fully parallelized.

For fine.txt, the dataset contained many small trees, resulting in mixed behavior. Both implementations saw gradual decreases in execution time as more workers were added, but the worker-pool version performed more efficiently overall, avoiding excessive goroutine creation. The two synchronization modes (channel vs. lock) behaved similarly at higher worker counts, where synchronization overhead dominated.

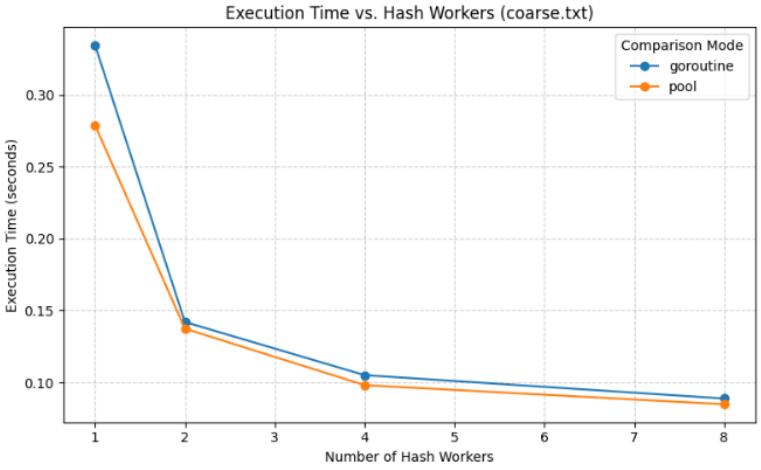
In summary, adding workers improved performance for larger datasets, particularly for coarse.txt, while simple.txt remained limited by concurrency overhead. The goroutine implementation generally achieved faster results, though the worker pool offered more predictable scaling for workloads with many small tasks.

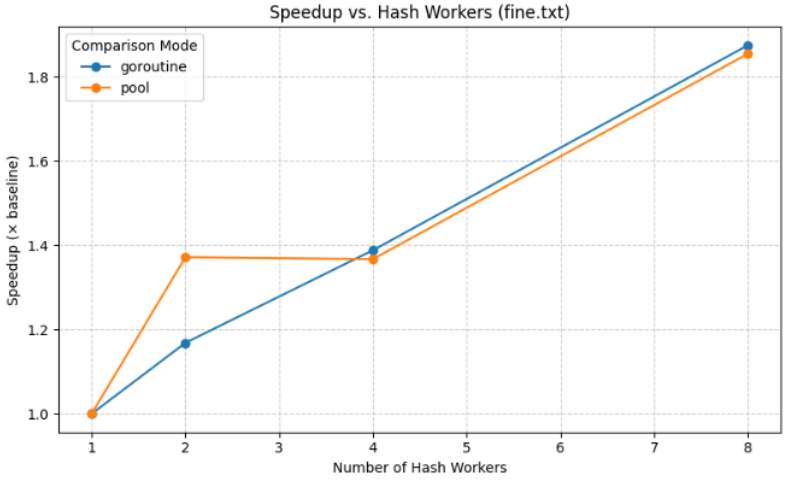
```
================== RESULTS ==================

Input file: input/simple.txt
------------------------------------------------------------
DataMode    CompMode      HashWorkers  Time
------------------------------------------------------------
channel     goroutine     1            701.7000 µs
channel     goroutine     2            1.4388   ms
channel     goroutine     4            733.3000 µs
channel     goroutine     8            0.0000   s
channel     pool          1            685.7000 µs
channel     pool          2            0.0000   s
channel     pool          4            0.0000   s
channel     pool          8            714.2000 µs
lock        goroutine     1            686.3000 µs
lock        goroutine     2            691.4000 µs
lock        goroutine     4            616.8000 µs
lock        goroutine     8            700.6000 µs
lock        pool          1            702.7000 µs
lock        pool          2            547.2000 µs
lock        pool          4            1.8787   ms
lock        pool          8            697.8000 µs
------------------------------------------------------------
```

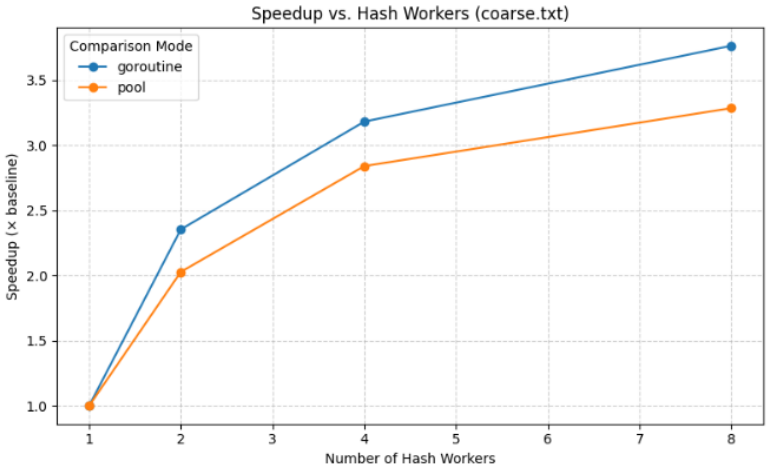
```
Input file: input/fine.txt
------------------------------------------------------------
DataMode    CompMode      HashWorkers  Time
------------------------------------------------------------
channel     goroutine     1            772.6701 ms
channel     goroutine     2            678.6044 ms
channel     goroutine     4            714.7180 ms
channel     goroutine     8            831.1271 ms
channel     pool          1            837.3118 ms
channel     pool          2            732.6413 ms
channel     pool          4            743.6583 ms
channel     pool          8            813.5739 ms
lock        goroutine     1            848.9405 ms
lock        goroutine     2            715.9175 ms
lock        goroutine     4            733.0701 ms
lock        goroutine     8            758.9362 ms
lock        pool          1            926.8854 ms
lock        pool          2            700.4283 ms
lock        pool          4            721.7020 ms
lock        pool          8            785.2449 ms
------------------------------------------------------------
============================================================
```

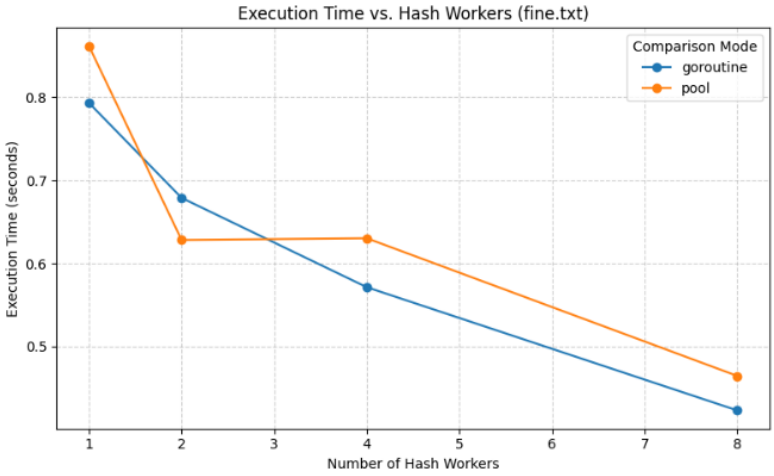
```
Input file: input/coarse.txt
------------------------------------------------------------
DataMode    CompMode      HashWorkers  Time
------------------------------------------------------------
channel     goroutine     1            261.3911 ms
channel     goroutine     2            235.3372 ms
channel     goroutine     4            272.1734 ms
channel     goroutine     8            283.5530 ms
channel     pool          1            291.5078 ms
channel     pool          2            226.8271 ms
channel     pool          4            208.2377 ms
channel     pool          8            289.9116 ms
lock        goroutine     1            245.8095 ms
lock        goroutine     2            243.4066 ms
lock        goroutine     4            221.5878 ms
lock        goroutine     8            224.1447 ms
lock        pool          1            265.7617 ms
lock        pool          2            233.7993 ms
lock        pool          4            211.8038 ms
lock        pool          8            267.3377 ms
------------------------------------------------------------
```

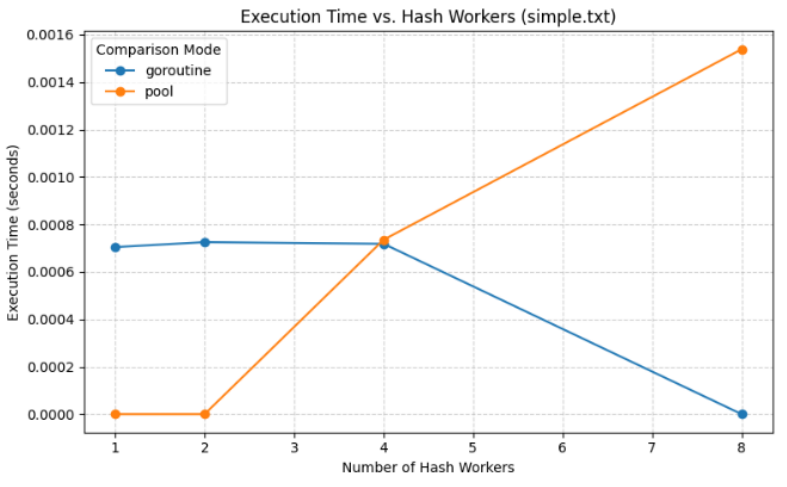
```
============================================================
channel     goroutine     8            831.1271 ms
channel     pool          1            837.3118 ms
channel     pool          2            732.6413 ms
channel     pool          4            743.6583 ms
channel     pool          8            813.5739 ms
lock        goroutine     1            848.9405 ms
lock        goroutine     2            715.9175 ms
lock        goroutine     4            733.0701 ms
lock        goroutine     8            758.9362 ms
lock        pool          1            926.8854 ms
lock        pool          2            700.4283 ms
lock        pool          4            721.7020 ms
lock        pool          8            785.2449 ms
channel     goroutine     8            831.1271 ms
channel     pool          1            837.3118 ms
channel     pool          2            732.6413 ms
channel     pool          4            743.6583 ms
channel     pool          8            813.5739 ms
lock        goroutine     1            848.9405 ms
lock        goroutine     2            715.9175 ms
lock        goroutine     4            733.0701 ms
lock        goroutine     8            758.9362 ms
lock        pool          1            926.8854 ms
channel     goroutine     8            831.1271 ms
channel     pool          1            837.3118 ms
channel     pool          2            732.6413 ms
channel     pool          4            743.6583 ms
channel     pool          8            813.5739 ms
lock        goroutine     1            848.9405 ms
lock        goroutine     2            715.9175 ms
lock        goroutine     4            733.0701 ms
channel     goroutine     8            831.1271 ms
channel     pool          1            837.3118 ms
```


Execution Time vs. Hash Workers (coarse.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
0.30
0.25
0.20
0.15
0.10
1
2
3
4
5
6
7
8
Number of Hash Workers


Speedup vs. Hash Workers (fine.txt)
Comparison Mode
goroutine
pool
Speedup (× baseline)
1.8
1.6
1.4
1.2
1.0
1
2
3
4
5
6
7
8
Number of Hash Workers


Speedup vs. Hash Workers (coarse.txt)
Comparison Mode
goroutine
pool
Speedup (× baseline)
3.5
3.0
2.5
2.0
1.5
1.0
1
2
3
4
5
6
7
8
Number of Hash Workers


Execution Time vs. Hash Workers (fine.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
0.8
0.7
0.6
0.5
1
2
3
4
5
6
7
8
Number of Hash Workers


Execution Time vs. Hash Workers (simple.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
0.0016
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
1
2
3
4
5
6
7
8
Number of Hash Workers

Takeaways:

For all three stages of the pipeline—hashing, hash grouping, and tree comparison—the sequential version serves as the performance baseline. When parallelizing the hash computation, I expected nearly linear speedup at low thread counts, since each tree can be hashed independently with no communication or shared data. This expectation mostly held true for large trees (coarse input), where the hashing work dominates and overhead is small. However, for many small trees (fine input), the benefit dropped off quickly because goroutine scheduling and channel or mutex costs outweighed the work being done. During the hash-grouping stage, I expected the mutex version to scale better than the channel-collector model, but the results showed the opposite for low and medium levels of parallelism: the channel version initially performed better because it avoided lock contention, although it later became a bottleneck as more workers were added. The mutex approach avoided the single-threaded choke point, but produced only modest speedups due to synchronization overhead. For the tree comparison step, I expected the worker-pool approach to outperform spawning a goroutine per comparison, and this was confirmed on larger inputs: the pool reduced scheduling overhead and delivered smoother scaling. Overall, all parallel approaches beat the sequential baseline on coarse workloads, but only hashing saw near-linear speedup. Grouping and comparison were limited more by coordination costs than by computation, which explains why their performance gains were smaller than originally expected.

Time Spent: 22 hrs